Question: "What is the difference between the `any` and `unknown` types in TypeScript?"

Analysis: This question tests a candidate's understanding of TypeScript's type system, specifically their ability to distinguish between flexibility and safety in handling values of uncertain types. It evaluates whether the candidate can explain the practical implications of using any versus unknown and articulate why type safety is important in modern software development.

Answer: The `any` and `unknown` types in TypeScript both represent values whose type is not known in advance, but they differ significantly in how they interact with the type system and enforce type safety.

The `any` type allows assignment to any type and permits any operation on the value without type checking. Essentially, it disables TypeScript's type checks for that variable, making it behave like plain JavaScript. Because of this, using `any` should be done sparingly, as it can lead to runtime errors and reduces the benefits of static typing.

The `unknown` type also allows assignment from any type, but does not permit any operation on the value without first narrowing or asserting its type. This forces explicit type checks before the value can be used, making code safer by preventing unintended operations and helping to avoid common runtime mistakes. Listing 3.14 shows some examples of `any` and `unknown`.

Listing 3.14 Examples of any and unknown

```
let vAny: any = 10;
let vUnknown: unknown = 10;
let s1: string = vAny; #A
vAny.method(); #B
let s2: string = vUnknown; #C
vUnknown.method(); #D
if (typeof vUnknown === "number") {
  let s3: string = vUnknown.toString(); #E
}
```

When processing deeply nested or highly dynamic data from third-party APIs, using `unknown` requires exhaustive type guards that can become cumbersome and impractical, especially if the data structure is unpredictable